

Assignment

Link Shortener API with Caching, Validation & Rate Limiting

Objective

Build a backend service that shortens URLs and tracks click analytics. The goal isn't just "make it work", it's to practice proper architecture, caching strategy, validation, rate limiting, and logging the way you'd need to in a real production service.

Prerequisites

Before starting, make sure you're comfortable with what you documented yesterday:

- Docker: containers, images, docker-compose, networking between services
- Redis: basic commands (GET, SET, INCR, EXPIRE, SET NX)
- Zod: schema definition, `.parse()` vs `.safeParse()`, `.refine()`, `.partial()`

If anything from your notes feels shaky, review it before diving in — you'll need all of it here.

Tech Stack

- Language: TypeScript
- Framework: Express
- Database: PostgreSQL
- Cache: Redis
- Validation: Zod
- Logging: pino
- Testing: supertest + your test runner of choice (jest if applicable)
- Containerization: Docker + docker-compose

Core Endpoints

Method	Route	Purpose
POST	/urls	Create a short link
GET	/:code	Redirect to original URL (tracks a click)
GET	/urls/:code/stats	Return click count, created date, last accessed
PATCH	/urls/:code	Update expiry/alias
GET	/urls	List links (paginated, filterable)
GET	/health	Liveness check
GET	/ready	Readiness check (Postgres + Redis reachable)

Folder Structure

Create the project with this structure from the start — don't let files pile up in one folder and reorganize later. Understanding why each folder exists is part of the assignment.

```
link-shortener/
├── docker-compose.yml
├── Dockerfile
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
├── README.md
├── src/
│   ├── index.ts          # app entrypoint — starts server, wires shutdown handlers
│   ├── app.ts            # express app setup, middleware registration
│   ├── config/
│   │   ├── env.ts        # parse & validate env vars (yes, with Zod)
│   │   ├── db.ts         # postgres client/pool setup
│   │   └── redis.ts       # redis client setup
│   ├── routes/
│   │   ├── url.routes.ts
│   │   └── health.routes.ts
│   ├── controllers/
│   │   └── url.controller.ts # request/response handling only — no business logic here
│   ├── services/
│   │   ├── url.service.ts  # core business logic: create, redirect, stats
│   │   ├── cache.service.ts # cache-aside, negative cache, stampede lock helpers
│   │   └── clickTracker.service.ts # redis INCR + interval flush to postgres
│   ├── repositories/
│   │   └── url.repository.ts # all postgres queries live here — nowhere else
│   ├── middlewares/
│   │   ├── validate.middleware.ts # generic zod-validation wrapper for routes
│   │   ├── rateLimiter.middleware.ts
│   │   ├── requestLogger.middleware.ts # assigns correlation id, logs each request
│   │   └── errorHandler.middleware.ts # catches typed errors, formats responses
│   ├── schemas/
│   │   └── url.schema.ts     # zod schemas: create, update, params
│   ├── errors/
│   │   ├── AppError.ts      # base error class
│   │   ├── NotFoundError.ts
│   │   ├── ValidationError.ts
│   │   └── RateLimitError.ts
│   ├── utils/
│   │   └── logger.ts         # pino instance + child-logger helper
│   ├── tests/
│   │   └── url.test.ts
│   └── migrations/
│       └── 001_create_urls_table.sql
```

Rule of thumb while building: if you're not sure where a piece of code goes, ask "what is this file's one job?" A controller's job is to parse the request and call a service — not query the database. A service's job is business logic — not knowing about req/res. A repository's job is talking to Postgres — nothing else.

Data Model

migrations/001_create_urls_table.sql:

```
CREATE TABLE urls (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  code VARCHAR(20) UNIQUE NOT NULL,  
  original_url TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT now(),  
  expires_at TIMESTAMP,  
  click_count BIGINT DEFAULT 0,  
  last_accessed_at TIMESTAMP  
);
```

Task Breakdown

Work through these in order — each phase builds on the last, and skipping ahead usually means redoing work.

Phase 1 — Project setup

- Initialize TS + Express project, set up folder structure above
- docker-compose.yml with app, postgres, redis services, healthchecks on postgres/redis
- Env validation with Zod in config/env.ts — the app should refuse to start if required env vars are missing/malformed

Phase 2 — Core CRUD (no caching yet)

- POST /urls — validate with Zod, generate a short code, save to Postgres
- GET /:code — look up Postgres directly, redirect, increment click_count directly in Postgres (optimized in Phase 3)
- GET /urls/:code/stats
- Get this working end-to-end and tested manually before adding caching — don't build cache logic on top of a shaky base

Phase 3 — Caching

- Cache-aside pattern on GET /:code
- Negative caching for invalid codes
- Cache stampede protection with a Redis lock
- Move click counting to Redis (INCR) with a periodic flush job to Postgres
- Cache invalidation on update/delete

Phase 4 — Validation depth

- PATCH /urls/:code with a .partial() schema
- Reserved-word blocklist for custom aliases
- expiresAt must be after createdAt via .refine()

Phase 5 — Rate limiting

- Redis-backed limiter middleware
- Tiered limits (anonymous vs API-key header)
- Proper 429 response with Retry-After header

Phase 6 — Logging & observability

- pino setup, correlation ID per request
- Log cache hit/miss, rate-limit rejections
- /health and /ready endpoints

Phase 7 — Resilience

- Centralized error handler with typed errors
- Graceful shutdown (SIGTERM → stop accepting requests → flush counters → close connections)

Phase 8 — Tests

- Integration test: create a link → redirect → confirm cache populated → check stats reflect the click

Requirements Checklist

- ☐ Layered architecture (controller/service/repository separation respected — no shortcuts)
- ☐ Cache-aside + negative cache + stampede protection implemented
- ☐ Click counts buffered in Redis, flushed to Postgres periodically
- ☐ Zod validation on all inputs (body, params, query)
- ☐ Rate limiting with tiered limits and proper headers
- ☐ Structured logs with correlation IDs
- ☐ docker-compose up brings up the full stack with healthchecks
- ☐ Graceful shutdown implemented
- ☐ At least one integration test passing

Bonus (only if core is solid and you have time)

- Pagination on GET /urls
- API-key based auth tier for rate limiting
- Multi-stage Dockerfile

Submission

- Push to a GitHub repo, share the link
- README should include: setup instructions (docker-compose up), a short explanation of your caching strategy in your own words, and anything you'd improve with more time
- Come prepared to walk through why you made specific decisions (e.g. TTL values, lock timeout) not just that they work.
- Submission Date: Jul 31, 2026 [11:00 AM] in office premises.